Enunciado S11 · RAG avanzado: Corrective RAG (CRAG) y comparación con RAG básico

Sesión: S11 · RAG avanzado (martes 27 de octubre de 2026, 18:30–20:00 `Europe/Madrid`).
Dedicación orientativa del alumno: 6 a 8 horas (práctica autónoma fuera de los 90 minutos de clase).
Objetivo: mejorar el control de la recuperación mediante reordenamiento (reranking), valoración explícita de suficiencia y rutas de corrección en LangGraph, y compararlo rigurosamente frente al RAG básico de S10 sin asumir que mayor complejidad siempre mejora la calidad.

---

1. Requisitos técnicos obligatorios del ejercicio

1.1 Base reconstruible desde índice vacío y conservación del RAG básico

- El proyecto debe ser **reconstruible desde ocho o más documentos ficticios en bruto e índice vacío**: lectura, limpieza, fragmentación con solapamiento, embeddings de **768 dimensiones** y almacenamiento/búsqueda en **PostgreSQL + `pgvector`** (`vector(768)` e índice HNSW).
- Configurar todos los modelos mediante variables de entorno (`GEN_MODEL_ID`, `EMBEDDING_MODEL_ID`, `RERANKER_MODEL_ID` / `EVALUATOR_MODEL_ID`, `DATABASE_URL`) y mantener **identificadores estables** de documentos y fragmentos.
- **Conservar operativo el RAG básico de S10 dentro del mismo proyecto** para ejecutarlo como línea base en la comparación.

1.2 Grafo en LangGraph con pasos separados (`retrieve` → `rerank` → `assess`)

Construir un grafo de estados en **LangGraph** (en Python o en LangGraph.js) que separe explícitamente las responsabilidades en nodos distintos:

1. **Recuperación inicial (`retrieve`):** - Recuperar hasta **diez candidatos** (`top_k = 10`) desde PostgreSQL/pgvector a partir de la consulta activa.
2. **Reordenamiento (`rerank`):** - Reordenar los candidatos y asignar a cada fragmento una puntuación en una **escala definida** (por ejemplo `[0, 1]`). - Puede utilizarse opcionalmente **`Qwen3-Reranker-0.6B`** con su formato oficial de prompt (`Judge whether the Document meets the requirements... Note that the answer can only be "yes" or "no"`) o un reordenador estructurado configurado por variable. - **Explicar la escala e interpretación de la puntuación:** una puntuación alta en un fragmento indica relevancia respecto a una parte de la consulta, pero **no garantiza por sí sola que haya cobertura suficiente para responder a toda la pregunta**.
3. **Valoración de suficiencia (`assess`):** - Nodo separado del reordenamiento que evalúa si los fragmentos útiles cubren la pregunta completa según **umbrales calibrados con ejemplos de validación** (distinguiendo **relevancia individual de un fragmento** y **cobertura global de la pregunta**). - Clasifica el estado de la evidencia y abre **tres rutas condicionales**:
    - **Evidencia suficiente (`sufficient`)** → pasar al nodo de generación (`answer`) usando únicamente los fragmentos aceptados en el orden validado.
    - **Evidencia ambigua o parcial (`partial`)** → conservar solo los fragmentos útiles (`retain`), formular una nueva consulta enfocada en la laguna pendiente (`reformulate`) y combinar la nueva búsqueda **sin duplicados**.

- **Evidencia insuficiente (`insufficient`)** → descartar el contexto rechazado de esa pasada (`discard`), reformular la consulta (`reformulate`) y volver a recuperar (`retrieve`).

4. **Límite estricto de iteración y rechazo (`refuse`):** - Permitir como máximo **dos reformulaciones después de la búsqueda inicial: tres recuperaciones internas en total**. - Si tras agotar las tres recuperaciones (o si la reformulación repite una consulta previa) sigue faltando evidencia suficiente, terminar en **rechazo controlado (`refuse`) sin llamar al modelo generativo**.

1.3 Registro de trazas y demostración de rutas

- Registrar en cada ejecución: la pregunta original (que nunca se sobrescribe), las consultas de búsqueda de cada intento, las puntuaciones de recuperación y reordenamiento, los fragmentos conservados y descartados con su intento y motivo, el veredicto de suficiencia y la ruta final.
- Demostrar con trazas verificables: 1. La ruta de **evidencia suficiente** en la primera búsqueda. 2. La ruta de **evidencia parcial/ambigua** (conservando un fragmento útil y recuperando el que faltaba en la segunda búsqueda sin duplicar). 3. La ruta de **evidencia insuficiente** con descarte del contexto irrelevante. 4. Un **rechazo y prueba controlada del límite** de 3 búsquedas y 2 reformulaciones.

---

2. Comparación ejecutada sobre 20 preguntas nuevas (14 respondibles + 6 no respondibles)

- Preparar **20 preguntas nuevas** (distintas de las de S10 y de los ejemplos de desarrollo/validación usados para diseñar el grafo y calibrar los umbrales):
- **14 preguntas respondibles** con apoyo en el corpus (incluyendo preguntas que requieran conectar más de una sección o documento).
- **6 preguntas no respondibles** (ausentes en los documentos).
- **Condiciones de comparación justa:**
- Congelar la configuración antes de ejecutar el lote.
- Usar exactamente el **mismo corpus, el mismo índice `vector(768)`, el mismo modelo generativo, las mismas instrucciones base y los mismos parámetros**, en conversaciones limpias.
- **Ejecutar ambos sistemas (RAG básico y CRAG) y presentar los resultados:**
- Informar en una tabla comparativa y en el detalle por caso:
  - **Aciertos en preguntas respondibles con numerador y denominador (`X/14`)**.
  - **Rechazos correctos en preguntas sin respuesta con numerador y denominador (`Y/6`)**.
  - **Recuperaciones medias por pregunta** (y reformulaciones medias en CRAG).
  - **Latencia** (mediana y media).
  - **Revisión manual de las fuentes citadas** frente a los documentos esperados.
- **Conservar y analizar los fallos:** no se exige usar Ragas ni demostrar que CRAG gane obligatoriamente al RAG básico. Una comparación sin ejecutar puede describirse honestamente si hubo un bloqueo, pero **no equivale a entregar la comparación ejecutada que pide el ejercicio**.
- **Recomendado, no obligatorio:** además del acierto, medir dos cosas que explican dónde aporta CRAG aunque empate en `X/14` e `Y/6`:

- **Margen de decisión:** en RAG básico, la similitud más baja del documento correcto en las respondibles frente a la similitud más alta en las no respondibles; en CRAG, lo mismo con la puntuación de `rerank`. Cuanto menor sea la distancia, más depende el resultado de un umbral exacto.
- **Contexto que llega al generador:** cuántas respuestas recibieron fragmentos obsoletos, de otra categoría o irrelevantes.

2.1 Referencia del laboratorio: un empate es un resultado válido

El laboratorio del repositorio (`lab/`, tienda ficticia **Zocomar**: 12 políticas, `vector(768)` + HNSW, Gemini Embedding 2 y Sonnet en las dos etapas) ejecutó este protocolo el 24 de octubre de 2026 con modelos reales:

Etapa	Respondibles	Rechazos correctos	Búsquedas medias	Latencia mediana (con respuesta)
RAG básico	14/14	6/6	1,00	3,2 s
CRAG	14/14	6/6	1,60	8,9 s

Empataron en acierto. La diferencia estuvo en el margen (0,014 de similitud en RAG frente a 0,65 de relevancia en el juez de CRAG) y en el contexto (RAG pasó una política sustituida al generador en 3 de 14 respuestas; CRAG, en ninguna), a cambio de casi tres veces más latencia. Tu resultado puede ser otro: lo que se evalúa es el método y la honestidad del análisis, no que CRAG gane. Evidencia: `docs/evidence/zocomar-lab-2026-10-24.md`.

---

3. Entrega

1. **Código completo** con el RAG básico de S10 y el grafo CRAG de S11 (con recuperación, reordenamiento, valoración, conservación/descarte, reformulación y respuesta/rechazo separados).
2. **Corpus sintético (8+ documentos) y script de reconstrucción desde índice vacío** en PostgreSQL/pgvector (`vector(768)` + HNSW).
3. **Trazas demostrativas** de las tres rutas (`sufficient`, `partial`, `insufficient`), del rechazo y del límite de 3 búsquedas / 2 reformulaciones.
4. **Resultados ejecutados de la comparación de 20 preguntas nuevas** entre RAG básico y CRAG (con `X/14`, `Y/6`, recuperaciones medias, latencia y revisión manual de fuentes).

(Opcional: búsqueda web pública como último recurso con fuentes marcadas y límite propio, nunca para inventar políticas internas ausentes).

---

4. Material de apoyo incluido en este repositorio

- **Laboratorio de referencia (`lab/`):** corpus Zocomar en `lab/corpus/sources/`; `lab/src/crag-graph.ts` (LangGraph con `retrieve` → `rerank` → `assess` → `retain`/`discard` → `reformulate` → `answer`/`refuse`); `lab/src/reranker.ts` (escala `[0, 1]`, plantilla de `Qwen3-Reranker-0.6B`, umbrales calibrados y `decideLabSufficiency`); `apps/api/src/lab-models.ts` (juez y

reformulador con un modelo real); `bun run lab:real compare` y `eval` para comparar con modelos reales. `lab/eval/resultados-laboratorio.md` es el respaldo determinista sin red: no mide modelos.

- **Guía de apoyo:** consulta `laboratorio-apoyo.md`.